

Name or Value? Cue Conflict Reveals When Chain of Thought Overrides Lexical Priors

Anonymous ACL submission

Abstract

Language models are misled by variable names that contradict a program’s behavior, but an accuracy drop cannot show where inside the model the name interferes. Interpretability studies find that intermediate values are computed during chain of thought, yet in their clean tasks the variable name and the computed value never disagree. We introduce a cue-conflict test for multi-step arithmetic in which each problem appears with congruent, neutral, and incongruent variable names under identical arithmetic, and evaluate two Llama models across four difficulty levels and three demonstration sets. Without a chain of thought the models read a variable’s value off its name: a congruent name raises accuracy by up to 20 points and an incongruent one yields the name’s value as the answer up to 20 points above chance. With a chain of thought the name’s value has no measurable effect on the answer at any level. Linear probes trained on neutral problems show that the lure is decodable only at the name’s own tokens and is erased at the step that writes the value; activation patching localises the no-chain lure to the early-layer representation of the equation that defines the name. Chain of thought thus overrides the lexical prior by replacing it, not by suppressing it. What a chain does not remove is a word-class effect on equation selection, and every effect we measure is smaller than the effect of which three demonstrations are shown.

1 Introduction

Consider a short arithmetic problem written with named variables:

pen=1 + cup, cup=2 + 3; pen=?

The answer is 6. Now rename the second variable to the word two:

pen=1 + two, two=2 + 3; pen=?

The arithmetic has not changed, and the answer is still 6. But the name now says something false. The word two denotes the number 2, while the variable it names holds the value 5. We call 2 the *lure value* of that name and 5 its *true value*. If a model answers with the lure, or uses it in a later step, we know the name’s meaning reached the computation.

This is the design used to separate reading from naming in the Stroop task (Stroop, 1935), where the word RED is printed in blue ink, and to separate shape from texture in vision models (Geirhos et al., 2019). We bring it to arithmetic.

Why it is needed here. Models read code and mathematics in which names carry meaning, and names that contradict the code hurt them: removing or corrupting identifiers degrades execution prediction (Le et al., 2025), misleading comments cut output prediction by about a fifth (Lam et al., 2025), and misleading renaming impairs comprehension in a study of humans and models alike (Le et al., 2026). None of that says *where* the name interferes. An accuracy drop is equally consistent with a corrupted input, a corrupted computation, and a correct computation misread at the output.

Interpretability on clean tasks cannot separate them either. Kudo et al. (2026) show, with linear probes and activation patching on exactly this kind of arithmetic, that the value of each variable becomes linearly decodable only as the chain of thought writes it. But their variable names are single letters. A decoded 5 can only have come from the computation, because no name ever says otherwise. Our incongruent names supply the disagreement, so a probe’s output tells us which source the representation holds.

We build matched sets: the same arithmetic in the naming conditions of Table 1 at the five difficulty levels of Table 2, solved with and without a written chain of thought, on two models. Our finding is that the chain of thought removes the

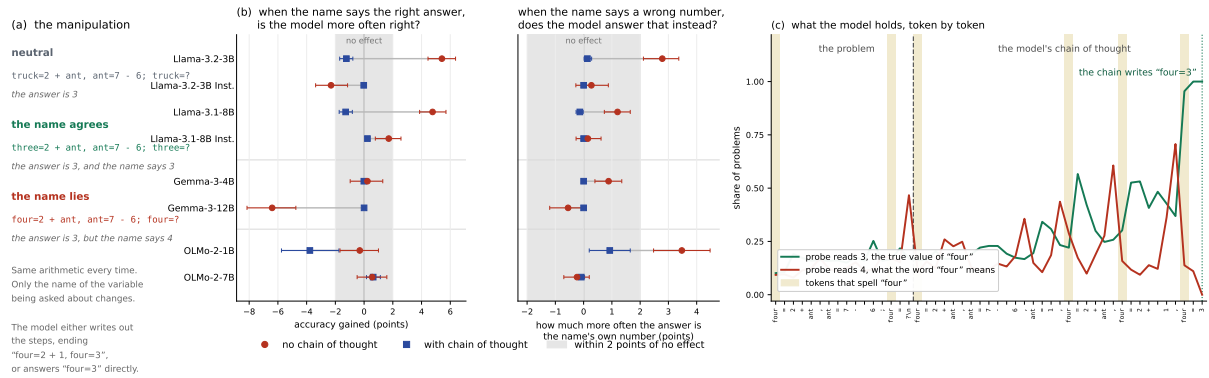


Figure 1: **(a)** The manipulation. One problem, three names for the variable the question asks about: a neutral word, a number word that states the answer, and a number word that states something else. The arithmetic and the answer never change. **(b)** What the name does to the answer, one row per model, grouped by family, at level 3, averaged over three sets of worked examples. Bars are 95% bootstrap intervals over problems; the grey line joins the two regimes for one model. Left: accuracy gained when the name states the answer. Right: how much more often the model answers the name’s own number than the neutral twin happens to give that same digit. Without a chain of thought (red) both effects are large. With a chain of thought (blue) every model falls inside the two-point band we fixed in advance as the smallest effect worth calling real. **(c)** Inside the model, at every token of one problem. A probe trained only on neutral problems reads 4, the number the name four states, through the problem and most of the chain; the true value 3 arrives only where the chain computes and writes it (dotted line). Until then the only number the state holds for that variable is the one its name says.

name’s influence by *writing over* it. Without a chain, models read a variable’s value off its name: a congruent name adds up to 20 points of accuracy, and an incongruent one supplies the answer up to 20 points above chance. With a chain, the name’s value has no measurable effect at any level or model. Probes show why: the lure is decodable only at the name’s own tokens, and it is gone at the step that writes the variable’s value.

Our contributions are:

- a cue-conflict version of a controlled arithmetic benchmark, with matched twins and generation rules that keep the lure from being correct for any other reason;
- a map of where the lure and the true value are decodable, by layer and by token, with and without a chain of thought;
- causal evidence, from patching activations between matched twins, for where the name’s influence lives when a chain is absent.

2 Setup

Task. We reuse the problems, difficulty levels and prompt format of Kudo et al. (2026). A problem is a list of equations over the digits 0–9 with + and –, ending in a query. Every variable’s value is itself a single digit. Three worked examples of the same level precede each problem. Table 2

gives one problem per level. Level 3 is our main level: two steps, one forward reference, and the level they analyse across models.

Naming conditions. Each problem is generated once and then rendered in the conditions of Table 1. Arithmetic, operators and digits are identical across a set; only names differ. In the *congruent* and *incongruent* conditions exactly one variable, the *target*, is named by a number word. Its value is the target’s true value in the congruent condition and something else, the *lure value*, in the incongruent one. The target is either the queried variable or the intermediate one, and we report the two separately, because a name on the variable you are asked about is not the same manipulation as a name on one you must compute on the way.

Generation rules. Only one variable per problem carries a number word. The lure differs from every variable’s value and from every digit in the problem, so a lure answer cannot be right for another reason. Names are single tokens in every context of the template under each model’s tokenizer, so twins have the same length and the same token positions (Appendix A). The *irrelevant lure* condition puts the number word on a distractor the answer does not depend on, separating the presence of a number word from a conflict at a variable the model must use.

Regimes. In the *chain-of-thought* regime the model writes the worked steps shown in Table 2 and then the answer. In the *direct* regime it writes only pen=6. The comparison between the two is the centre of the paper.

Demonstration sets. Which three worked examples are shown turns out to matter more than any name, so every behavioural number below is computed for three fixed demonstration sets and reported as their mean. We call an effect reliable only when its sign is the same in all three sets and a cluster bootstrap over matched problems excludes zero.

Probes. A linear probe reads a chosen variable’s value from the residual stream, one probe per position, layer and seed, with Kudo et al.’s recipe. Probes are trained on neutral problems only, so they never see a number word as a name; what they report on an incongruent problem is a property of the representation, not of the probe. Each is paired with a control probe trained on labels that are a fixed random function of the variable’s *name* (Hewitt and Liang, 2019). Where the control also scores highly, the probe is reading name identity rather than a value, and we report it as such.

Patching. To test whether a name causes a behaviour we replace the activations at its tokens with the matched twin’s and re-read the output (Zhang and Nanda, 2024). Two controls accompany every test: swapping one neutral word for another bounds the damage any swap does, and swapping one lure for a different lure shows whether the answer follows the name.

Models. Llama-3.1-8B, Llama-3.2-3B.

3 Results

Replication. On the letter condition our per-token probes reproduce Kudo et al.’s Table 3 for Llama-3.2-3B exactly: the queried variable becomes decodable at equation 5 of the chain and the intermediate at equation 2, with pre-chain maxima of 17.1 and 47.9 (theirs: 17.8 and 33.2).

RQ1: behaviour. Table 3 reports, per model and regime, accuracy and the pre-registered contrasts pooled over three demonstration sets, with a contrast claimed only when its sign holds in every set. Without a chain the name’s value is used as a shortcut: a congruent name on the queried variable adds 4.8 points for Llama-3.1-8B at level 3 and up to 20

Condition	Problem	True	Lure
Letter	A=1 + B, B=2 + 3; A=?	—	—
Neutral	pen=1 + cup, cup=2 + 3; pen=?	—	—
Congruent	pen=1 + five, five=2 + 3; pen=?	5	—
Incongruent	pen=1 + two, two=2 + 3; pen=?	5	2
Irrelevant (level 4)	pen=1 + cup, cup=2 + 3, two=4 + 5; pen=?	9	2

Table 1: The naming conditions, shown on one problem, whose answer is 6 in every row of the top block. The model is asked to solve it either by writing the chain pen=1 + two, two=2 + 3, two=5, pen=1 + two, pen=1 + 5, pen=6 or by writing pen=6 directly. *True* is the value of the variable that carries the number word; *Lure* is the value that word denotes. Each problem also has a twin with a different neutral word and a twin with a different lure, used as controls when patching.

Level	Problem (neutral names)	Steps	Held	Distr.
1	pen=1 + cup, cup=2; pen=?	1	1	0
2	pen=2 + 3, cup=1 + pen; cup=?	2	0	0
3	pen=1 + cup, cup=2 + 3; pen=?	2	1	0
4	pen=1 + cup, cup=2 + 3, box=4 + 5; pen=?	2	1	1
5	pen=1 + cup, cup=2 + box, box=1 + 2; pen=?	3	2	0

Table 2: The five levels of Kudo et al. (2026). *Steps* is how many additions or subtractions the answer needs. *Held* is how many variables cannot be resolved when first read. *Distr.* is how many equations the answer does not depend on. The chain of thought the model writes restates each equation, substitutes the values it has found, and writes each variable’s value in turn, as in cup=2 + 3, cup=5, pen=1 + cup, pen=1 + 5, pen=6. Level 1 is our no-computation control: the intermediate’s value is stated, not computed.

points at level 5, and an incongruent name yields lure answers 1.2 points above chance at level 3 and up to 20 points where accuracy is at floor. With a chain the same contrasts are within 0.7 points of zero in every cell, and congruent and incongruent names move together. The irrelevant-lure control at level 4 separates presence from binding: a number word on the distractor costs accuracy but produces no lure answers. What the chain does not remove is a word-class effect: a number-word name on the queried variable at level 4 costs Llama-3.2-3B 10.5 points whether or not its value agrees, and the error analysis shows the model restating the wrong equation next.

RQ2: representation. Figure 1(c) shows what the probe reads at every token, and Figure 2 in

Model	Regime	Neutral accuracy (%)	number word on the queried variable		number word on the intermediate variable	
			helps	lures	helps	lures
Llama-3.1-8B	CoT	95 [86,100]	-1.3	-0.1	+3.8	-0.0
Llama-3.1-8B	direct	53 [41,62]	+4.8*	+1.2*	+3.8*	-0.0
Llama-3.2-3B	CoT	98 [97,100]	-1.2	+0.1	+0.9	-0.2*
Llama-3.2-3B	direct	22 [13,31]	+5.4*	+2.8*	+1.1	+1.8

Table 3: Does the name’s value change what the model answers? Level 3, averaged over three demonstration sets, with the range across those sets in brackets. *Helps* is the accuracy gained when the number word matches the variable’s value (congruent minus neutral, in points). *Lures* is how much more often the model answers the number word’s own value than it would by chance (incongruent minus the rate at which the neutral twin gives that same digit). * marks an effect whose sign is the same in all three demonstration sets and whose bootstrap interval excludes zero. Every starred effect is in the direct rows. Other levels are in Table 4.

the appendix adds the layer sweep behind it. At P2, the end of the defining equation, the true value is already partly computed (accuracy 0.49, selectivity 0.32) and the lure is weak (lure rate 0.06). At P3, the query, a neutral-trained probe reads the lure in 32% of instances, but the control task scores 0.97 there: the readout is the name’s identity, not a value, and we do not claim it. At P4 and P5 the true value is decodable at 0.98 with the lure at zero in every condition and level; per-token probes show the lure decodable only at the name’s own tokens, and a number word bound to a computed variable has lure mass 0.007 at the answer against 0.118 for an unbound one on the distractor. The outcome cell is “lure fades, answer correct”, reached by replacement at the value step rather than suppression.

RQ3: causation. Figure 3 in the appendix plots, for the direct regime, the share of lure errors removed by patching the name’s tokens from the matched neutral twin, by layer, with both controls. For Llama-3.2-3B the neutral patch removes 52% of lure errors and restores the true-minus-lure margin fully (normalised logit difference 1.00), the alternative-lure patch is followed in 9% of cases, and the span grid places the effect in the defining equation at layers 0–7. The word control changes 22% of correct answers at this floor accuracy, so we report the localisation and not a recovery rate. For Llama-3.1-8B the no-chain lure rate at level 3 is at chance and both patches behave alike; the effect exists only where the lure does. With a chain there are no lure errors to patch and both controls are clean.

4 Related Work

Kudo et al. (2026) established when sub-answers are computed during chain of thought on clean arithmetic; we add cue conflict to ask what that computation is sensitive to. Two recent studies bring the Stroop design to language models: Wang (2026) pits a local redefinition of a word against its familiar associate and restores the lost margin by patching from a matched neutral prompt; Hu et al. (2026) read a congruency effect off next-token probabilities in a colour-rule task every model solves. Both measure one next token; neither has errors, a chain of thought, or probes over reasoning steps. Faithfulness has been measured from the outside by perturbing prompts, chains or parameters (Turpin et al., 2023; Lanham et al., 2023; Chen et al., 2025; Tutek et al., 2025); we measure it inside the model under a cue whose true and lure values are both known. On arithmetic, Liu (2026) show that small models copy the trailing chain number and Prabhakar et al. (2024) that an output prior can override a correct chain; our late-readout outcome is stated against a positional-copy control. Misleading identifiers and comments hurt on real code (Le et al., 2026, 2025; Lam et al., 2025); we trade realism for localisation. Binding work asks how a value attaches to a name (Feng and Steinhardt, 2024; Wu et al., 2025; Gur-Arieh et al., 2026); we ask whether the attachment survives a name carrying a competing value. Shi et al. (2023) add irrelevant context; here the distractor is the name itself. The design is Stroop (1935) and Geirhos et al. (2019).

5 Conclusion

A misleading variable name is used when the model has nothing better: without a chain of thought the name’s value becomes the answer, with a chain it is replaced at the step that computes the value and never reaches the output. For faithfulness this is the reassuring cell of the outcome table, and it comes with two cautions: the chain leaves a word-class effect on which equation the model attends to next, and the demonstration set moves accuracy more than any name.

Limitations

The task is synthetic arithmetic with short chains, so the results need not transfer directly to real code, which is the motivation and not the finding. Models are at most 8B parameters; within-family com-

parisons give the direction of the trend only. Lures are hand-picked English number words; digit-bearing names such as x2, other languages and optimised lures are future work. Linear probes can miss nonlinearly encoded information; the patching experiment partly addresses this, but hidden states carry mixed information and patching cannot rule out side effects entirely, which is why two control patches are reported. The chain-of-thought format writes the name beside the computed value at every step, which may itself aid recovery; the direct regime bounds that effect from the other side.

References

Yanda Chen et al. 2025. Reasoning models don’t always say what they think. *arXiv preprint arXiv:2505.05410*.

Jiahai Feng and Jacob Steinhardt. 2024. How do language models bind entities in context? In *International Conference on Learning Representations*.

Robert Geirhos, Patricia Rubisch, Claudio Michaelis, Matthias Bethge, Felix A. Wichmann, and Wieland Brendel. 2019. ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. In *International Conference on Learning Representations*.

Yoav Gur-Arieh, Mor Geva, and Atticus Geiger. 2026. Mixing mechanisms: How language models retrieve bound entities in-context. In *International Conference on Learning Representations*. ArXiv:2510.06182.

John Hewitt and Percy Liang. 2019. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*.

Xiaoyang Hu, Mike Angstadt, Shane Storks, Zan Huang, Aman Taxali, Alex Weigard, Richard L. Lewis, and Chandra Sripada. 2026. Conflict and congruency effects in large language models: In-weight and in-context competition in a verbal conflict task. *arXiv preprint arXiv:2608.11510*.

Keito Kudo, Yoichi Aoki, Tatsuki Kuribayashi, Shusaku Sone, Masaya Taniguchi, Ana Brassard, Keisuke Sakaguchi, and Kentaro Inui. 2026. LLMs faithfully and iteratively compute answers during CoT: A systematic analysis with multi-step arithmetics. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 1114–1153. Code: <https://github.com/keitokudo/faithful-cot-multistep-arithmetic>.

Man Ho Lam, Chaozheng Wang, Jen-tse Huang, and Michael R. Lyu. 2025. CodeCrash: Exposing LLM

fragility to misleading natural language in code reasoning. In *Advances in Neural Information Processing Systems*, volume 38. ArXiv:2504.14119.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamilė Lukošiušė, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, and 11 others. 2023. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*.

Cuong Chi Le, Minh V. T. Pham, Cuong Duc Van, Hoang N. Phan, Huy N. Phan, and Tien N. Nguyen. 2025. When names disappear: Revealing what LLMs actually understand about code. *arXiv preprint arXiv:2510.03178*.

Jack Le, Anh H. N. Nguyen, and Tien N. Nguyen. 2026. Do machines struggle where humans do? LLM and human comprehension of obfuscated code. *arXiv preprint arXiv:2606.31725*. Cite in the third person under [review].

Ming Liu. 2026. The readout shortcut: Positional number copying dominates arithmetic CoT readout in small language models. *arXiv preprint arXiv:2605.22870*.

Akshara Prabhakar, Thomas L. Griffiths, and R. Thomas McCoy. 2024. Deciphering the factors influencing the efficacy of chain-of-thought: Probability, memorization, and noisy reasoning. *arXiv preprint arXiv:2407.01687*. Venue to verify (likely Findings of EMNLP 2024).

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. 2023. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning*.

J. Ridley Stroop. 1935. Studies of interference in serial verbal reactions. *Journal of Experimental Psychology*, 18(6):643–662.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. 2023. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, pages 74952–74965.

Martin Tutek, Fateme Hashemi Chaleshtori, Ana Marasović, and Yonatan Belinkov. 2025. Measuring chain of thought faithfulness by unlearning reasoning steps. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Han-yu Wang. 2026. Persistent priors, preserved targets: A Stroop-style paradigm for lexical override. *arXiv preprint arXiv:2606.07555*.

Yiwei Wu, Atticus Geiger, and Raphaël Millière. 2025. How do transformers learn variable binding in symbolic programs? In *Proceedings of the 42nd International Conference on Machine Learning*.

Fred Zhang and Neel Nanda. 2024. Towards best practices of activation patching in language models: Metrics and methods. In *International Conference on Learning Representations*.

A Tokenizer checks

Rule 4 is decided per model, not by eye. For each model we tokenize every candidate name in the five contexts of the template (line start, after a comma, after each operator, before the separator) and keep only words that are a single token in all five with no merging into a neighbour. For the Llama 3 tokenizer, all ten number words pass and 95 of 112 neutral candidates pass; the lists are released with the code. Binary operators carry a space on each side because – otherwise merges with a following word (1–two tokenizes as 1, –two), which would break the alignment between twins. With the released lists, every instance of a condition tokenizes to the same length with the same position indices, and the runner excludes and counts any instance that does not; no instance was excluded in any run reported here.

B Full sweeps and selectivity

For every (variable, position, layer, seed) we report probe accuracy for the true value, lure rate, lure mass and margin, alongside the accuracy of a control probe trained on labels that are a deterministic random function of the variable’s name (He-witt and Liang, 2019). Selectivity is the difference. The query position is the one cell where a high lure rate coincides with control accuracy near 1, which is why it is reported as name identity rather than as a represented value. Per-token sweeps over the whole instance, with the max-over-layers curve, are given for both regimes and all three conditions, together with t^* , t_{eq}^* and the pre-/post-chain accuracies in Kudo et al.’s format.

C Additional levels, models and demonstration sets

Table 4 repeats Table 3 for the other four levels.

Per-set values, the pooled estimate and the claim rule are released with the code. The appendix also contains the span-by-layer patching

Model	Level	Regime	Neutral accuracy (%)	number word on the queried variable		number word on the intermediate variable	
				helps	lures	helps	lures
Llama-3.1-8B	1	CoT	99 [98,100]	+0.0	+0.0	+0.3	+0.0
Llama-3.1-8B	1	direct	74 [38,99]	+16.1*	+11.2*	+8.8*	+1.3
Llama-3.2-3B	1	CoT	97 [95,100]	+1.1	+0.1	-2.4	+0.3
Llama-3.2-3B	1	direct	46 [21,93]	+0.6	+5.9*	-3.9*	+2.6*
Llama-3.1-8B	2	CoT	100 [100,100]	-0.1	+0.0	-0.0	+0.0
Llama-3.1-8B	2	direct	36 [33,37]	+11.8*	+0.7	+2.6	+2.2
Llama-3.2-3B	2	CoT	94 [90,98]	+3.7	+0.5*	+1.0*	-0.1
Llama-3.2-3B	2	direct	29 [27,30]	+10.8*	+2.5*	+2.6*	+2.2
Llama-3.1-8B	4	CoT	88 [63,100]	+1.0	-0.0	-2.3	+0.2
Llama-3.1-8B	4	direct	28 [26,31]	+13.7*	+5.8	+6.0*	+0.2
Llama-3.2-3B	4	CoT	64 [26,85]	-10.9*	+0.7	-2.7*	+0.4
Llama-3.2-3B	4	direct	20 [16,23]	-1.6	+1.9	-0.2	+1.2*
Llama-3.1-8B	5	CoT	65 [43,100]	+1.2	-0.3	+0.1	-0.1
Llama-3.1-8B	5	direct	20 [13,26]	+20.2*	+20.5*	+0.9	+0.4
Llama-3.2-3B	5	CoT	69 [48,97]	-1.4	+0.1	-2.6	-0.1
Llama-3.2-3B	5	direct	12 [11,12]	+4.7*	+3.5*	+1.1*	+0.5

Table 4: As Table 3, for levels 1, 2, 4 and 5. Level 1 is the no-computation control: the intermediate variable’s value is stated rather than computed.

grids for all three sources, the level-4 irrelevant-lure control, the lure-distance analysis, the teacher-forced lure rates, and the error taxonomy that identifies the equation-selection failure behind the word-class effect.

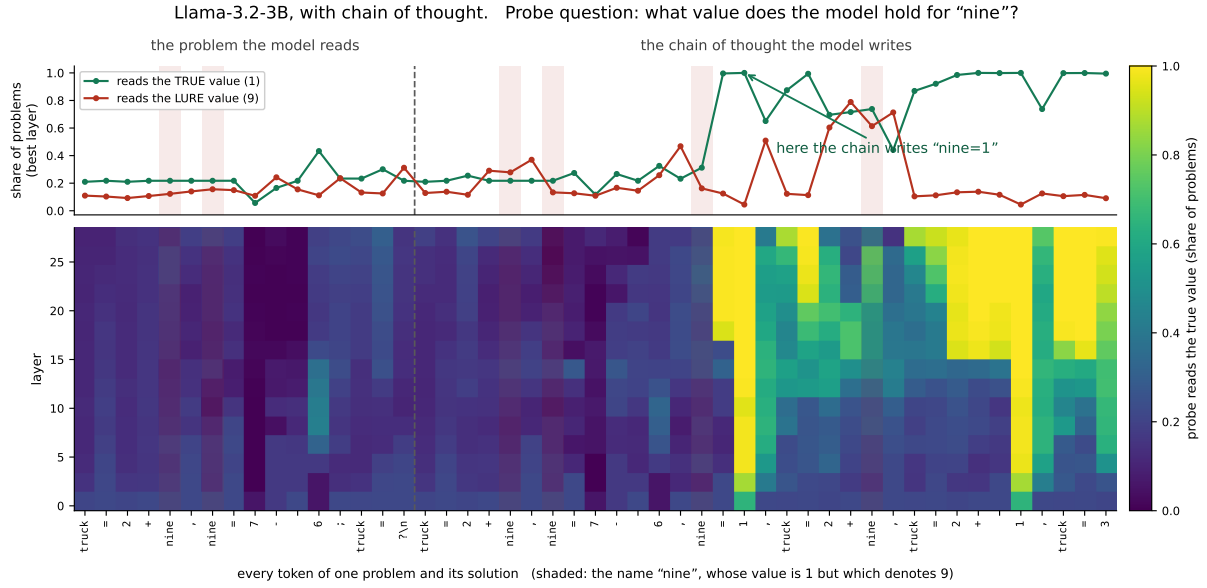


Figure 2: What the model holds for the variable whose name lies. The probe is asked, at every token, for the value of nine, a variable whose name denotes 9 but whose value is 1. **Top:** how often the probe’s answer is the true value (green) and how often it is the lure (red), taking the best layer at each token. **Bottom:** the same true-value accuracy for every layer. The problem is to the left of the dashed line and the model’s own chain of thought to the right; shaded columns are the tokens of the name itself. The lure is read only at those tokens and at the query, and it disappears the moment the chain writes `nine=1`, after which the true value is held for the rest of the solution. One problem’s tokens are shown; positions are the same for all 2,000.

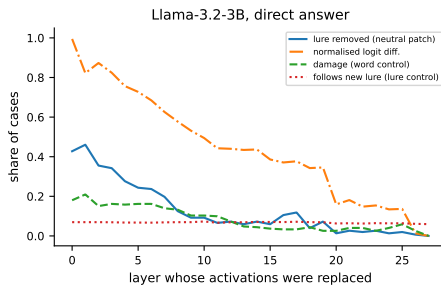


Figure 3: Lure errors removed by patching the name’s tokens from the neutral twin, by layer, direct regime, with the neutral-word control patch (damage) and the alternative-lure patch.